Phase 3: Estimators, controller and calibration

Fault tolerant control and post fault trajectory planning of a redundant cable driven parallel robot

M. J. Tachia, A. V. Maloletov

Technical report, 26 October 2026

About this document. This document records how the estimators and the controller of the methods (`docs/phase1/methods.pdf`, equations (M n)) were built, what had to change on the way and why, and how the settings were fixed on calibration runs. Nothing here is a result of the study: every number below comes either from development runs (seeds below 100) or from the calibration set (seeds from 100000), and the test set of Phase 5 will use seeds from 200000, so that the reported results never share a parameter draw, a noise sequence or an onset time with the data that shaped the controller. The frozen configuration is the file `configs/controller_phase3.yaml` (Section 8).

1 What was built

Table 1 lists the components and the rates at which they run in the 1 kHz loop of the simulator. Every component sees only what the controller of a real robot would see: the encoder angles and the motor torques of the eight winches, one sample late, the nominal parameters, the homing lengths and the reference. None of them reads the plant state.

Table 1: Components of the controller.

Component	Equations	Rate	Implementation
Winch observer (ESO) and tension loop	(M 22)–(M 25)	1 kHz	NumPy, vectorised over the winches
IMM, 17 modes, 28 states per filter	(M 27)–(M 33)	200 Hz	prediction and measurement models compiled to C, batched algebra
Probability aware MPC, up to 3 scenarios	(M 34)	50 Hz	IPOPT with exact Hessian, callbacks compiled to C
Planner after a break	(M 36), Sec. 9	on declaration	offline library along each task; online fallback
Actuator fault accommodation	Sec. 9	1 kHz	correction of the winch estimate by $\hat{\delta}_i$

The termination rule that Phase 2 left open is now fixed: a run counts as a lost platform, and stops, when a tilt angle exceeds 30°, when the reference point leaves the interior of the frame, or when the drum of a cable that is not broken leaves its window (M 15).

2 Winch level

The observer (M 22) is implemented in discrete time as a current estimator with all three poles at $e^{-\omega_o \Delta t}$, $\omega_o = 2\pi \cdot 50$ rad/s, and the gains are placed by Ackermann's formula; a test checks the placed poles to $2 \cdot 10^{-4}$. The tension loop (M 25) uses $k_P = 0.5$, $k_I = 20$ s^{-1} with the integral clipped at 20 N. When a break is declared, the winch of the broken cable is held at its angle by a proportional derivative law of 20 Hz bandwidth, so that it does not run away.

On the 16 fault free runs of the second calibration set, run again through the observers alone (`scripts/phase3_eso_check.py`), the estimate follows the cable tension of the plant with an RMS error of 0.30 N at the nominal current noise and 0.57 N at twice that noise (one sample of measurement delay accounted for). Compared with the tension 10 ms earlier the error falls to 0.23 and 0.46 N: three poles at $2\pi \cdot 50$ rad/s delay the estimate by $3/\omega_o = 9.5$ ms. At 100 Hz the delay halves and the error at the best alignment rises to 0.33 and 0.66 N, at 25 Hz the delay doubles; the replay of Section 3 shows what each of the three does to the diagnosis. The delay is also the reason the IMM takes the commands, and not these estimates, as the tensions acting on the platform (methods, Sec. 7.2).

3 Diagnosis

Changes to the methods. Four changes came out of this phase, and the methods now carry them (Sec. 7.2): bias states shared by all modes, homing of the length offsets, arming of the fault modes after

$t_{\mathrm{arm}} = 2$ s, and two numerical safeguards. The first version of the IMM had the twelve platform states and the deficit only. On randomised plants its $\mathcal{A}_i$ filters, whose deficit is a free parameter, absorbed the mismatch of mass and centre of mass and took the probability in fault free runs; the slowly varying wrench $\boldsymbol{w}_b$, the length offsets $\boldsymbol{\beta}$ and the stiffness ratio κ give that mismatch a place that a fault, being abrupt, cannot use. With the bias states, a fault mode still took over in all four fault free development runs within 0.13 s of the start, before the bias states had converged; with the fault modes armed at 1 s it happened in two of four, and with $t_{\mathrm{arm}} = 2$ s in none of six. After a break, the filters of the implausible modes see innovations of tens of newtons; one of them returned non finite values in a development run, which ended the run on the assertion that the probabilities sum to one, and the safeguards of the methods prevent that. An IMM cycle takes 2.3 ms on average (Section 4).

Settings from the calibration runs. The calibration runs are recorded with declarations disabled, so their mode probabilities are those of an undisturbed diagnosis. The winch observers and the IMM can then be run again offline on the recorded readings and commands with other settings (`cdpr_ft/estimation/diagnosis.py`; the controller and the replay share the same class, and the replay with the recorded setting reproduces the logged probabilities to $3 \cdot 10^{-8}$, the rounding of their storage). For another setting the replay keeps the plant motion of the recorded run. The probabilities act on the motion only once a fault mode exceeds $\mu_s = 10^{-3}$, where the MPC adds its scenario, so the replay is exact up to that point and approximate beyond it. Table 2 gives the result of changing one setting at a time on the second calibration set (the one with the final MPC weights).

Table 2: Replay of the diagnosis on the 56 runs of the second calibration set, one setting changed at a time. Fault free time: 453 s (the fault free runs and the parts of the fault runs before the onset). Delays are medians, from the onset to the first time the true mode reaches 0.99. Wrong first: break runs in which another fault mode exceeded 0.9 before the true one. LOE: loss of effectiveness.

	Fault free data		Breaks (8)		LOE (16)	Bias (16)
Setting changed	largest fault μ	time above 0.5 [s]	delay [ms]	wrong first	delay [ms]	delay [ms]
none (first setting)	0.998	1.28	9	4	10	20
σ_a (linear) $0.5 \to 0.25$	1.000	3.47	9	4	10	20
σ_a (linear) $0.5 \to 1$	0.998	0.88	9	4	10	21
σ_α (angular) $0.5 \to 0.25$	1.000	35.67	9	4	10	18
σ_α (angular) $0.5 \to 1$	0.361	0.00	9	4	10	24
r_t $1.5 \to 0.75$ N	0.998	1.28	9	5	10	20
r_t $1.5 \to 3$ N	0.998	1.28	9	4	10	20
r_l $0.2 \to 0.1$ mm	1.000	9.82	9	4	8	14
r_l $0.2 \to 0.4$ mm	0.999	2.58	9	2	14	31
fault rate $10^{-3} \to 10^{-4}$ s^{-1}	0.996	0.68	9	4	10	23
fault rate $10^{-3} \to 10^{-2}$ s^{-1}	0.998	2.24	9	4	10	20
σ_δ $20 \to 5$ N s$^{-1/2}$	1.000	36.65	9	3	11	26
σ_δ $20 \to 50$ N s$^{-1/2}$	0.946	0.12	9	5	10	20
σ_β $10^{-5} \to 10^{-4}$ m s$^{-1/2}$	0.989	0.87	9	4	10	21
σ_w (force) $0.5 \to 2$ N s$^{-1/2}$	1.000	0.96	9	4	10	23
t_{arm} $2 \to 1$ s	0.998	1.67	9	4	10	20
t_{arm} $2 \to 1.5$ s	0.998	1.65	9	4	10	20
observer 50 $\to$ 25 Hz	0.978	0.72	19	8	13	25
observer 50 $\to$ 100 Hz	1.000	2.81	4	0	8	19
Second round						
$\sigma_\alpha = 2$	0.139	0.00	9	4	10	29
$\sigma_\alpha = 1$, observer 100 Hz	0.996	0.28	4	0	8	21
$\sigma_\alpha = 2$, observer 100 Hz	0.991	0.27	4	0	8	29

The rule for this choice was written before the replay was run: keep the first setting unless another one, with every fault reaching 0.99 and no wrong mode above 0.9 before the true one, lowers the largest fault mode probability in the fault free data by a factor of ten. No setting met the condition, the first one included, for a reason the rule had not foreseen: after a break, the actuator mode of the same winch leads for a single estimator cycle in about half of the runs before the break mode takes over, and the declaration rule, which needs $N_c \geq 2$ consecutive cycles, never acts on it. Read literally, the rule keeps the first setting. The table shows two settings with a strong effect, however: the angular process noise, which at 1 rad s^{-2} removes every fault mode probability above 0.5 from the fault free data at the price of 4 ms on the median bias delay, and the observer bandwidth, which at 100 Hz halves the break delay and removes the wrong first cycle but raises

the fault free excursions. A second round combined the two, with a rule fixed before it was run: among the settings without any fault mode probability above 0.5 in the fault free data, the fewest wrong first cycles, then the smallest median break delay, then the smallest median bias delay. Both settings with the observer at 100 Hz fail the first condition, and among the remaining two the smaller bias delay selects $\sigma_\alpha = 1\ \mathrm{rad\,s^{-2}}$ with the observer at 50 Hz. We adopted it, and the third calibration set was run with it, so that the thresholds of Section 6 come from closed loop runs of the final estimator and not from the replay. That was necessary. In one run (a loss of effectiveness on winch 5, onset at 6.33 s) an actuator mode of another winch rises to 0.99 about half a second before the onset with the first setting. Replayed with $\sigma_\alpha = 1\ \mathrm{rad\,s^{-2}}$ the same readings give at most 0.36, but run in closed loop with that setting the excursion reaches 0.98: once the MPC weighs the actuator scenario, the motion it produces makes that scenario more plausible. The probability aware MPC can thus amplify a transient doubt of the diagnosis. In the third calibration set this run is the only one, out of 453 s of fault free operation, in which a fault probability exceeds 0.01 before the onset. We record the amendment here because a rule that is changed after its outcome is seen has to be reported as such.

What the replay also shows. In 5 of the 16 bias runs of the second calibration set, the true actuator mode, which takes the lead between 21 and 49 ms after the onset, later loses it to the actuator mode of another winch: in four of them less than 0.1 s after the onset, in the fifth after 1.3 s. With the thresholds of Section 6 the declaration comes before that in every case of the calibration set, but bias faults of 10% are the hardest case of the diagnosis, and their isolation has to be checked in the confusion matrix of Phase 5. Without declarations, which these runs disable, a loss of effectiveness of 60% on an upper winch lost the platform 0.6 to 1.3 s after the onset in all four such runs, and on a lower winch in none; this shows why accommodation is needed, and is not a defect of the diagnosis.

4 Model predictive control

Implementation. The programme (M 34) is written in CasADi [Andersson et al., 2019] with the step of the prediction model and the stage cost mapped over the horizon, so that the generated code holds one copy of each per step function, and it is solved by IPOPT [Wächter and Biegler, 2006] with the exact Hessian, from the solution of the previous sample shifted by one step. One solver is compiled for each number of scenarios (one to three); the C code of each is about 15 MB and compiles in five minutes, once. An expanded scalar version of the same programme produced 79 MB of code, and a limited memory Hessian took 114 ms per solve without converging, so neither was kept. The weights are parameters of the compiled programme. The horizon is $N = 12$ steps of 20 ms; the MPC also supplies the drum accelerations $\ddot{\theta}^{\mathrm{ff}}$ of (M 25), from the second difference of the cable lengths along its predicted motion.

Weights. The first weights penalised position and orientation errors with $4 \cdot 10^4$ per m^2 and rad^2 (an error of 5 mm costs as much as one of 5 mrad), velocity errors with 400, and tension changes between samples with 0.01 per N^2. With them the closed loop oscillated and lost the platform before any fault in 7 of the 56 runs of the first calibration set, and tracked with errors above 10 mm in two more. All seven lost runs had cables 15 to 20% softer than nominal (EA drawn between 0.80 and 0.85 of its nominal value), and every other run had a ratio of at least 0.86. The oscillation shows in the tensions: in one of these runs the lower cables went from 40 N to slack and back within 40 ms, half a second after the start, while the commands stepped by up to 37 N between control samples. Softer cables lower the elastic modes of the platform, which the prediction model leaves out, toward the bandwidth of the closed loop. Three scalings of the velocity and tension change weights were compared on the 16 fault free calibration runs (first 8 s of each, which hold the start transient and the earliest onsets), with the rule, fixed in advance, to take the setting with no lost platform and the smallest median tracking error (`scripts/phase3_mpc_weights.py`).

Table 3: MPC weights on the 16 fault free calibration runs (0 to 8 s). Error: RMS distance between the reference point and its reference.

Setting	Velocity weights	Tension change weight	Lost	Error, median / max [mm]
First	400	0.01	2	2.75 / 104.8
A	40	0.1	0	2.64 / 5.16
B (chosen)	120	0.03	0	2.57 / 5.39
C (A, pose weights halved)	40	0.1	0	2.68 / 5.22

The three reduced settings differ by less than 0.1 mm in the median, so the choice among them hardly matters for tracking; it matters for the margin to the oscillation, which B, the closest to the first setting, has

least of. The full calibration set run with B is the check of that margin: in the second and third calibration sets and in the final check (Section 7), 168 runs in all, no platform was lost before a fault.

After a break. The velocity weights are multiplied by 0.3 and the tension change weight by 3 once a break is declared, which brings them to 36 and 0.09, close to setting A. With the first weights kept after a break, four development breaks gave 40 to 72% of the post fault time with a tension out of bounds and one lost platform; with the velocity weights divided by ten and the tension change weight multiplied by ten, 0 to 5% and none lost. The elastic modes of Section 5 give the reason. The scaling was checked on the eight breaks of the calibration set, run with declarations until 4 s after the onset, against keeping the weights unchanged, with the rule, fixed before the runs, to prefer fewer lost platforms and then the smaller median share of post fault time with a tension out of bounds (`results/phase3_check_post_break_*.json`). With the scaling no platform was lost, the median share was 1.6% (at most 18.3%) and the largest tension of an intact cable 271 N; without it one platform was lost, the median share was 3.4% (at most 72%) and the largest tension 544 N.

Computation. In the final check of Section 7, with two runs sharing the two cores of the machine (Python 3.13, CasADi 3.8.1, IPOPT with MUMPS), an MPC solve took 24 ms on average, against a period of 20 ms; the 99th percentile within a run reached 94 ms and the longest solve 238 ms. An IMM cycle took 2.3 ms on average (period 5 ms), with a 99th percentile of up to 8.4 ms. These times do not affect the simulation, in which the controller is called synchronously, but they mean that this implementation would not run in real time on this machine (Section 9).

5 Planner

Safe sets. For every cable, the poses that the seven remaining cables hold with a margin of at least $s_{\min} = 10$ N were computed on a grid of 1859 positions ($x, y \in [-1.5, 1.5]$ m, $z \in [1.5, 4.0]$ m, step 0.25 m), trying orientations of a 10° grid up to 20° about each axis in order of increasing tilt, with every remaining cable inside its drum window (`scripts/phase3_safe_poses.py`). After the loss of an upper cable 620 positions hold, 21% of them at zero orientation, with a median margin of 15.1 to 15.5 N; after the loss of a lower cable 1141 positions hold, 42% at zero orientation, with a median margin of 17.4 to 18.3 N. The sets seed the recovery problem and supply the terminal targets of the fault scenarios of the MPC.

Why the quintic recovery path was replaced. The methods of Phase 1 joined the estimated state to the nearest safe pose with a quintic polynomial. After an upper cable breaks on T1, at $z = 3$ m, no duration between 0.4 and 3 s made that path dynamically feasible: along the best candidate the smallest margin was between -31 and -78 N in the cases checked, with the shortfall in the middle of the descent, not at its start. Letting the polynomial start with the acceleration the platform actually has did not help. The seven cables can only produce accelerations inside a narrow set that changes with the pose, and a polynomial of fixed shape does not stay in it. Branch 2 is therefore the optimal control problem (M 36), whose dynamics are the rigid model itself. Two versions of its cost were tried before the one in the methods. Minimum time with the tensions free inside their bounds converged in a median of 43 and up to 146 IPOPT iterations and gave paths with the tensions on their bounds and the yaw swinging three times in a second; tracked in closed loop, those paths left no tension in reserve for the feedback. The fixed duration of 1 s with the twist and acceleration terms and the soft margin of 20 N gives smooth paths and needs at most 45 iterations.

Time scaling. Scaling the time of the remaining reference from the instant of the break makes its velocity jump from $\boldsymbol{\nu}$ to $\kappa\boldsymbol{\nu}$. The time scale now changes from 1 to κ along a quintic ramp of 0.5 s, and the check of the dynamic margin includes the deceleration of the ramp; the derivatives of the slowed reference are verified by a test against finite differences. The bisection on κ stops at $\kappa_{\min} = 0.25$: in the first version it went down to $\kappa = 0.001$, which keeps the platform on the path but does not continue the task in any useful sense.

Latency and the library. The recovery problem is solved in 0.21 s (median), 0.38 s (99th percentile) and at most 0.82 s on one core of the machine used here (Table 4). During that time the platform falls. In development runs on the nominal plant, with the MPC gains already reduced after the break, four breaks (cables 1, 2, 5 and 7) were run with the plan applied 0.1, 0.3 and 0.5 s after the declaration, starting from the state the MPC predicts for that instant and following the nominal path until then. The share of post fault time with the tension of an intact cable outside $[t_{\min}, t_{\max}]$ was 0 to 5% at 0.1 s, 9 to 14% at 0.3 s and 15 to 40% at 0.5 s, and from 0.3 s on the recovery problem of the cable 1 break was no longer solvable within

the bounds. Steering toward the nearest safe pose before the plan starts, instead of following the nominal path, did not remove the effect: it helped after the lower cable breaks (1 to 7% at 0.3 s) and made the upper cable breaks worse, up to a lost platform at 0.5 s. Because the reference paths of a task are known before the task runs, the planner is therefore evaluated in advance: `scripts/phase3_recovery_library.py` computes, for T1 and T2 and every cable, the branch and the plan for break instants every 50 ms from 2.5 to 8.0 s, which covers the onsets of the specification (3 to 7 s) with room for the detection delay. On a declaration at t_d, the controller takes the plan of the break instant nearest to $t_d - 15$ ms (the typical delay of a break declaration, Section 6) and follows it from t_d on; the oracle baseline takes the instant of the onset itself. The online solution, applied after a latency of 0.4 s that covers the 99th percentile of the solve times, remains available for states the library does not cover and for an ablation.

Table 4: Recovery library: 111 break instants per cable and path, T1 and T2 together (222 per cable). Branch 1 is the time scaling, branch 2 the recovery problem; branch 3 (descent) never occurred. Drop: largest fall of the reference point below its height at the break. Largest angle: largest Euler angle along the plan. IPOPT needed at most 45 iterations.

Cable lost	Branch 1 / 2	Smallest κ (branch 1)	Drop, median / max [m]	Largest angle [deg]	Solve time, median / max [s]
1	0 / 222	–	0.60 / 0.88	25.0	0.21 / 0.61
2	0 / 222	–	0.60 / 0.95	25.0	0.21 / 0.43
3	0 / 222	–	0.54 / 0.88	25.0	0.20 / 0.36
4	0 / 222	–	0.54 / 0.95	25.0	0.20 / 0.36
5	31 / 191	0.26	0.00 / 0.02	23.6	0.21 / 0.45
6	36 / 186	0.26	0.00 / 0.02	23.5	0.23 / 0.82
7	69 / 153	0.26	0.00 / 0.02	21.9	0.24 / 0.43
8	35 / 187	0.25	0.00 / 0.02	21.9	0.21 / 0.42

Over all 1605 plans of branch 2 the local error of the collocation (the largest position error over the 15 intervals against an RK4 integration at 1 ms from each node with the tension of the interval) is at most 0.12 mm, and the open loop drift is at most 0.66 mm after 0.25 s but reaches 0.11 m at the end of a recovery after an upper cable break, against 4 mm after a lower cable break. The drift does not shrink when the number of intervals is doubled (development check on three cases), so it is not a discretisation error: the platform hanging from seven cables along these paths is unstable on its own, and the plan has to be tracked in closed loop.

Elastic modes after a break. At the end poses of the 1605 recovery plans, with their holding tensions and locked winches, the lowest elastic mode of the platform lies between 3.2 and 3.6 Hz, against 5.5 Hz at the home pose with eight cables (Phase 1); the highest lies below 18.3 Hz (`scripts/phase3_postfault_modes.py`). This is the reason for the lower MPC gains after a break (Section 4).

6 Declaration thresholds

The declaration rule of the methods (Sec. 7.2) needs two numbers: the probability μ_{iso} a fault mode must reach and the number N_c of consecutive estimator cycles it must keep it. Because the calibration runs are recorded with declarations disabled, and nothing in the controller depends on the thresholds before a declaration, the first declaration that any pair would make can be read from the recorded probabilities exactly (`scripts/phase3_roc.py`). Table 5 gives part of the grid (μ_{iso} from 0.5 to 0.9999, N_c from 1 to 8) on the third calibration set.

Every bias of the calibration set is declared correctly at every pair shown, and each further confirmation cycle adds 5 ms to every delay. Two things decide the choice. With $N_c = 1$ one break is declared as the wrong mode, the single cycle in which the actuator mode of the same winch leads (Section 3). Below $\mu_{\text{iso}} = 0.99$ the excursion of run 38 becomes a false alarm. The rule, written before the grid was computed and amended once (`scripts/phase3_roc.py`): among the pairs without false alarm and with every fault declared correctly, the smallest median break delay, ties broken toward the larger μ_{iso} and N_c. As first written, the rule did not require correct isolation and picked $N_c = 1$; the condition, which was its intent, was added after that result was seen. It selects $\mu_{\text{iso}} = 0.9999$ and $N_c = 2$: on the calibration set a break is declared after 15 ms (at most 18 ms), a loss of effectiveness after 23 ms (34 ms) and a bias after 42 ms (99 ms), with no false alarm. The largest fault mode probability before an onset was 0.979, so the margin to a false alarm at this threshold is set by one run, and 453 s of fault free operation say nothing about the false alarm rate per hour, which Phase 5 measures on the test set.

Table 5: Declaration thresholds on the third calibration set (final estimator and MPC; 16 fault free runs and 40 fault runs, 453 s of fault free operation). FA: runs with a declaration before the onset. Correct: declarations of the true mode after the onset, out of the runs without false alarm. Delays from the onset to the declaration, median / max.

μ_{iso}	N_c	FA	Breaks correct	Break delay [ms]	LOE correct	LOE delay [ms]	Bias delay [ms]
0.9	1	1	7/8	9 / 13	15/15	15 / 26	32 / 83
0.9	2	1	8/8	15 / 18	15/15	20 / 31	37 / 88
0.99	1	0	7/8	9 / 13	16/16	15 / 29	33 / 89
0.99	2	0	8/8	15 / 18	16/16	20 / 34	38 / 94
0.999	2	0	8/8	15 / 18	16/16	21 / 34	40 / 94
0.9999	**2**	**0**	**8/8**	**15 / 18**	**16/16**	**23 / 34**	**42 / 99**
0.9999	3	0	8/8	20 / 23	16/16	28 / 39	47 / 138
0.9999	4	0	8/8	25 / 28	16/16	33 / 44	52 / 143

7 Closed loop check on the calibration set

The frozen configuration was run on all 56 specifications of the calibration set with declarations enabled, the fault runs until 4 s after the onset (`scripts/phase3_closed_loop_check.py`). These runs are calibration data, and their only purpose is to confirm that the frozen controller does what the calibration was meant to achieve; they are not results of the study. Table 6 summarises them.

Table 6: Frozen configuration on the calibration set. Delay: onset to declaration, median / max. Out of bounds: share of the post fault time in which the tension of an intact cable is outside $[t_{\min}, t_{\max}]$, median / max over the runs. Error: RMS tracking error of the fault free runs.

Runs	False alarms	Correct	Delay [ms]	Lost	Out of bounds [%]	Largest tension [N]
Fault free (16)	0	–	–	0	–	–
Breaks (8)	0	8/8	16 / 19	0	1.6 / 18.3	271
Loss of effectiveness (16)	0	16/16	24 / 35	0	0.0 / 3.0	187
Bias (16)	0	16/16	43 / 100	0	0.0 / 0.0	173

In the fault free runs the RMS tracking error was 2.9 mm (median) and at most 5.8 mm. Every fault was declared as the true mode, every break was answered by a recovery plan from the library (all eight onsets fell where the time scaling was not feasible), and no platform was lost; the four losses of effectiveness of 60% on an upper winch, which lost the platform when the fault was not handled (Section 3), were all held by the accommodation. The weakest case is the break of cable 3 at 5.38 s on T1: a tension of an intact cable was out of bounds during 18% of the 4 s after the onset, mostly slack (0.68 s with a slack cable), the tilt reached 26.7°, 3.3° short of the termination rule, the largest tension was 271 N, and the platform came to rest 11 mm from the end of its plan. In the other seven breaks the share stayed at or below 5% and the platform came to rest within 8 mm of the end of its plan.

8 Frozen configuration

The controller of the study is now fixed by four files, which Phase 5 must use unchanged:

- `configs/controller_phase3.yaml`, every setting of the estimators, the MPC, the planner, the recovery problem and the decision logic, with the source of each group of values in its header (SHA-256 prefix `71c8e8640f29365f`);
- `results/phase3_recovery_library.npz`, the plans along T1 and T2 (`a8749c838681c808`);
- `results/phase3_safe_poses.npz`, the safe sets (`994fb7d2aaa623a3`);
- the code, with the digest `fe7722d2d710c537`.

From this freeze on, the code digest covers the package and the robot configuration, and every result file carries the hash of the controller configuration next to it; the result files of Phase 3 written before the freeze carry the digest of the code at their time. The final check of Section 7 was run with exactly these files. It was run twice, before and after the provenance change (the configuration files of the two runs differ only in the

header comment), and the second run reproduced every outcome of the first exactly: declarations, delays, losses, tensions and errors; only the computation times differ. Table 7 lists the settings that were chosen or changed in this phase; the configuration file holds the complete list.

Table 7: Settings fixed in Phase 3 and where they come from. Design: chosen and stated, not tuned.

Group	Setting	Value	Source
Observer and loop	bandwidth $\omega_o/2\pi$; k_P, k_I	50 Hz; 0.5, 20 s^{-1}	design; replay (Table 2)
IMM	rate; arming t_{arm}	200 Hz; 2 s	timing; development runs
	σ_a, σ_α	0.5 $\mathrm{m\,s}^{-2}$, 1 $\mathrm{rad\,s}^{-2}$	design; replay (second round)
	r_l, r_t, σ_δ	0.2 mm, 1.5 N, 20 $\mathrm{N\,s}^{-1/2}$	design, unchanged by the replay
	fault rate, return rate	10^{-3}, 10^{-2} s^{-1}	design, unchanged by the replay
	innovation gate	10^4	safeguard
Declaration	μ_{iso}, N_c	0.9999, 2	ROC (Table 5)
MPC	period, horizon	20 ms, 12 steps	design
	weights: pose; velocity; tension change	$4 \cdot 10^4$; 120; 0.03	Table 3
	after a break	velocity $\times 0.3$, tension change $\times 3$	development runs; calibration check
	scenarios: threshold μ_s, at most	10^{-3}, 2 fault scenarios	design
Planner	T_h, $\kappa_{\min}$, ramp T_r	2 s, 0.25, 0.5 s	design
Recovery (M 36)	T, intervals; s_p, s_r, $s_{\min}$	1 s, 15; 5, 20, 10 N	design
	library grid; detection delay	50 ms, 2.5 to 8 s; 15 ms	design; Table 5
	online latency (fallback)	0.4 s	99th percentile of solve times
Termination	tilt; frame; drum window	30°; interior; intact cables	design

9 Carried into the next phases

1. *Computation time.* On this machine the MPC does not meet its 20 ms period on average when two runs share the two cores (Section 4). The simulation is not affected, because the controller is called synchronously, but the solve times have to be reported against the period, together with what a real time implementation would need (a structure exploiting solver, a fixed iteration budget). The campaign estimate of Phase 2 rises accordingly: a run of the full controller costs 3.9 to 4.9 s of computation per simulated second here (two runs in parallel on the two cores), so a 16 s run takes 60 to 80 s instead of the 25 to 65 s estimated in Phase 2, and the 4100 runs per method and condition of the specification take about 80 core hours.

2. *The library assumes a known task.* The plans are computed in advance along T1 and T2. The online planner remains for other states, with its latency of 0.4 s, and should enter Phase 5 as an ablation, since it shows what the library buys.

3. *Bias isolation.* Biases of 10% are declared correctly on the calibration set, but the diagnosis loses the true mode afterwards in some runs (Section 3); the confusion matrix of Phase 5 will show how often this costs a correct declaration on the test set.

4. *Amplification by the hedging MPC.* One calibration run showed a transient doubt of the diagnosis that the MPC scenarios amplified, without a false alarm at the chosen threshold. The false alarm rate per hour on the test set, and the comparison with the variant without hedging (ablation A1), will tell whether this matters.

5. *Seeds.* Development runs used seeds below 100, the calibration set seeds from 100000. The test set of Phase 5 must use seeds from 200000, and no setting may change after this freeze; a change forces a new freeze and a new calibration check.

6. *Phase 4.* The learned classifier (baseline B3) will be trained on runs generated with this frozen configuration, from its own seeds (300000 and up), so that its training data share nothing with the calibration and test sets.

References

Joel A. E. Andersson, Joris Gillis, Greg Horn, James B. Rawlings, and Moritz Diehl. CasADi: a software framework for nonlinear optimization and optimal control. *Mathematical Programming Computation*, 11 (1):1–36, 2019. doi: 10.1007/s12532-018-0139-4.

Andreas Wächter and Lorenz T. Biegler. On the implementation of an interior-point filter line-search algorithm for large-scale nonlinear programming. *Mathematical Programming*, 106(1):25–57, 2006. doi: 10.1007/s10107-004-0559-y.